

**DEPSTAR**  
Department of Computer Science & Engineering  
CSPIT, CHARUSAT

# Java Interfaces

## Comprehensive Class Notes

---

### **Topics Covered:**

- Introduction to Interfaces
- Declaring and Implementing Interfaces
- Multiple Inheritance with Interfaces
- Default, Static, and Private Methods (Java 8/9)
- Extending Interfaces
- Best Practices and Use Cases

**Academic Year:** 2026-267  
**Course:** Object-Oriented Programming  
**Prepared by:** Mr. Ronak Sir and Team

August 13, 2026

## Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>What is an Interface?</b>                          | <b>2</b>  |
| 1.1      | Key Characteristics . . . . .                         | 2         |
| 1.2      | CHARUSAT-CSPIT Use Case Example . . . . .             | 2         |
| <b>2</b> | <b>Declaring and Implementing Interfaces</b>          | <b>3</b>  |
| 2.1      | Syntax and Structure . . . . .                        | 3         |
| 2.2      | Important Rules . . . . .                             | 4         |
| 2.3      | CSPIT Example: Laboratory Equipment System . . . . .  | 4         |
| <b>3</b> | <b>Multiple Inheritance Using Interfaces</b>          | <b>5</b>  |
| 3.1      | Why Interfaces Support Multiple Inheritance . . . . . | 5         |
| 3.2      | CSPIT Example: Student Activities . . . . .           | 6         |
| <b>4</b> | <b>Java 8 Features in Interfaces</b>                  | <b>7</b>  |
| 4.1      | Default Methods . . . . .                             | 7         |
| 4.2      | Static Methods in Interfaces . . . . .                | 9         |
| 4.3      | Functional Interfaces . . . . .                       | 9         |
| <b>5</b> | <b>Java 9+ Features: Private Methods</b>              | <b>10</b> |
| 5.1      | Private Methods in Interfaces . . . . .               | 10        |
| <b>6</b> | <b>Extending Interfaces</b>                           | <b>12</b> |
| 6.1      | Interface Inheritance . . . . .                       | 12        |
| 6.2      | CSPIT Example: Department Hierarchy . . . . .         | 12        |
| <b>7</b> | <b>Best Practices and Use Cases</b>                   | <b>15</b> |
| 7.1      | When to Use Interfaces . . . . .                      | 15        |
| 7.2      | CSPIT Example: Integrated University System . . . . . | 15        |
| <b>8</b> | <b>Summary and Key Takeaways</b>                      | <b>18</b> |
| 8.1      | Important Points to Remember . . . . .                | 18        |
| 8.2      | CSPIT Implementation Checklist . . . . .              | 18        |
| 8.3      | Common Mistakes to Avoid . . . . .                    | 19        |

# 1 What is an Interface?

An **interface** in Java is a **blueprint** that defines a set of methods a class must implement without providing full implementation details. It helps achieve **abstraction** by focusing on **what** a class should do rather than **how** it does it.

## 1.1 Key Characteristics

- **100% Abstraction:** All methods are abstract by default (before Java 8)
- **Multiple Inheritance:** Interfaces allow a class to inherit from multiple sources
- **Contract Definition:** Specifies what a class should do but not how
- **Variable Constants:** All variables in an interface are **public**, **static**, and **final** by default

**Important:** Starting from Java 8, interfaces can also contain **default** and **static** methods with implementation.

## 1.2 CHARUSAT-CSPIT Use Case Example

In our university system, we can define an AcademicEntity interface that outlines the essential behaviors for all academic entities:

```
1 // Interface defining a contract for all academic entities at
  CHARUSAT
2 interface AcademicEntity {
3
4     // Public, static, and final (constant)
5     String UNIVERSITY_NAME = "CHARUSAT";
6     String DEPARTMENT = "Computer Science & Engineering";
7
8     // Abstract methods - to be implemented by classes
9     void displayDetails();
10    double calculateGrade();
11    boolean isEligibleForScholarship();
12 }
13
14 // Student class implementing AcademicEntity
15 class CSPITStudent implements AcademicEntity {
16     private String name;
17     private String studentId;
18     private double cgpa;
19
20     public CSPITStudent(String name, String studentId, double
      cgpa) {
21         this.name = name;
22         this.studentId = studentId;
23         this.cgpa = cgpa;
24     }
25 }
```

```
26     @Override
27     public void displayDetails() {
28         System.out.println("Student Name: " + name);
29         System.out.println("Student ID: " + studentId);
30         System.out.println("Department: " + DEPARTMENT);
31         System.out.println("University: " + UNIVERSITY_NAME);
32     }
33
34     @Override
35     public double calculateGrade() {
36         // Simplified grade calculation
37         return cgpa * 10; // Convert CGPA to percentage
38     }
39
40     @Override
41     public boolean isEligibleForScholarship() {
42         return cgpa >= 9.0; // CHARUSAT scholarship criteria
43     }
44 }
45
46 public class UniversitySystem {
47     public static void main(String[] args) {
48         CSPITStudent student = new CSPITStudent("Rahul Sharma", "
49             CSPIT2025001", 9.2);
50         student.displayDetails();
51         System.out.println("Grade: " + student.calculateGrade() +
52             "%");
53         System.out.println("Scholarship Eligible: " + student.
54             isEligibleForScholarship());
55     }
56 }
```

Listing 1: AcademicEntity Interface for CHARUSAT

## 2 Declaring and Implementing Interfaces

### 2.1 Syntax and Structure

An interface is declared using the **interface** keyword:

```
1 // Interface Declaration
2 interface TestInterface {
3     // Public, static, final variable
4     final int a = 10;
5
6     // Public, abstract method
7     void display();
8 }
9
10 // Class implementing the interface
11 class TestClass implements TestInterface {
```

```
12 // Implementing the interface method
13 public void display() {
14     System.out.println("Hello from CSPIT");
15 }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         TestClass t = new TestClass();
21         t.display();
22         System.out.println("Value of a: " + t.a);
23     }
24 }
```

Listing 2: Basic Interface Declaration

## 2.2 Important Rules

1. A class **must implement** all abstract methods of the interface
2. All variables in an interface are **public**, **static**, and **final** by default
3. Interfaces can have **default**, **static**, and **private** methods (Java 8/9)
4. To implement an interface, use the **implements** keyword
5. A class can implement multiple interfaces

## 2.3 CSPIT Example: Laboratory Equipment System

```
1 interface LabEquipment {
2     String LAB_NAME = "CSPIT Computer Lab";
3
4     void powerOn();
5     void powerOff();
6     boolean isOperational();
7 }
8
9 class DesktopComputer implements LabEquipment {
10     private boolean operational = true;
11
12     @Override
13     public void powerOn() {
14         System.out.println("Desktop Computer is booting up...");
15     }
16
17     @Override
18     public void powerOff() {
19         System.out.println("Desktop Computer is shutting down...");
20     }
21 }
```

```
22     @Override
23     public boolean isOperational() {
24         return operational;
25     }
26 }
27
28 class Projector implements LabEquipment {
29     private boolean operational = true;
30
31     @Override
32     public void powerOn() {
33         System.out.println("Projector is warming up...");
34     }
35
36     @Override
37     public void powerOff() {
38         System.out.println("Projector is cooling down...");
39     }
40
41     @Override
42     public boolean isOperational() {
43         return operational;
44     }
45 }
46
47 public class LabSystem {
48     public static void main(String[] args) {
49         DesktopComputer pc = new DesktopComputer();
50         Projector projector = new Projector();
51
52         System.out.println("--- " + LabEquipment.LAB_NAME + " ---");
53         pc.powerOn();
54         projector.powerOn();
55
56         System.out.println("PC Operational: " + pc.isOperational());
57         System.out.println("Projector Operational: " + projector.isOperational());
58     }
59 }
```

Listing 3: Lab Equipment Interface System

## 3 Multiple Inheritance Using Interfaces

### 3.1 Why Interfaces Support Multiple Inheritance

- Java doesn't support multiple inheritance with classes to avoid ambiguity (Diamond Problem)

- Interfaces solve this by having no method implementations (before Java 8)
- With default methods, ambiguity is resolved by explicit overriding

### 3.2 CSPIT Example: Student Activities

```
1 // Interface for Academic Activities
2 interface AcademicActivities {
3     void attendClass();
4     void submitAssignment();
5     double calculateAttendance();
6 }
7
8 // Interface for Sports Activities
9 interface SportsActivities {
10    void participateInSport();
11    void attendPractice();
12    int getSportsScore();
13 }
14
15 // Student class implementing both interfaces
16 class CSPITStudent implements AcademicActivities,
    SportsActivities {
17     private String name;
18     private int attendance = 85;
19     private int sportsScore = 75;
20
21     public CSPITStudent(String name) {
22         this.name = name;
23     }
24
25     // Academic Activities Implementation
26     @Override
27     public void attendClass() {
28         System.out.println(name + " is attending CSPIT class");
29     }
30
31     @Override
32     public void submitAssignment() {
33         System.out.println(name + " has submitted assignment");
34     }
35
36     @Override
37     public double calculateAttendance() {
38         return attendance;
39     }
40
41     // Sports Activities Implementation
42     @Override
43     public void participateInSport() {
44         System.out.println(name + " is participating in sports");
```

```
45     }
46
47     @Override
48     public void attendPractice() {
49         System.out.println(name + " is at practice");
50     }
51
52     @Override
53     public int getSportsScore() {
54         return sportsScore;
55     }
56
57     public void displayFullProfile() {
58         System.out.println("Student: " + name);
59         System.out.println("Attendance: " + attendance + "%");
60         System.out.println("Sports Score: " + sportsScore);
61     }
62 }
63
64 public class StudentActivitiesDemo {
65     public static void main(String[] args) {
66         CSPITStudent student = new CSPITStudent("Priya Patel");
67         student.attendClass();
68         student.submitAssignment();
69         student.participateInSport();
70         student.attendPractice();
71         student.displayFullProfile();
72     }
73 }
```

Listing 4: Multiple Inheritance in Student Activities

## 4 Java 8 Features in Interfaces

### 4.1 Default Methods

- Allow adding new methods to interfaces without breaking existing implementations
- Provide default implementation that can be overridden by implementing classes

```
1 interface CSPITCourse {
2     String COURSE_NAME = "Java Programming";
3     int CREDITS = 4;
4
5     void teachContent();
6     void conductExam();
7
8     // Default method - new feature in Java 8
9     default void provideStudyMaterials() {
10         System.out.println("Providing standard study materials
            for " + COURSE_NAME);
```

```
11     System.out.println("Includes: Lecture slides, practice
12         questions, and reference books");
13 }
14 // Static method - Java 8 feature
15 static void displayCourseInfo() {
16     System.out.println("Course: " + COURSE_NAME);
17     System.out.println("Credits: " + CREDITS);
18     System.out.println("Department: Computer Science &
19         Engineering");
20 }
21 }
22 class JavaProgrammingCourse implements CSPITCourse {
23     @Override
24     public void teachContent() {
25         System.out.println("Teaching Java fundamentals including
26             OOP concepts");
27     }
28     @Override
29     public void conductExam() {
30         System.out.println("Conducting written and practical exam
31             ");
32     }
33     // Optionally override default method
34     @Override
35     public void provideStudyMaterials() {
36         System.out.println("Providing custom study materials with
37             additional coding exercises");
38     }
39 }
40 public class CourseDemo {
41     public static void main(String[] args) {
42         // Static method call
43         CSPITCourse.displayCourseInfo();
44
45         JavaProgrammingCourse course = new JavaProgrammingCourse
46             ();
47         course.teachContent();
48         course.conductExam();
49         course.provideStudyMaterials(); // Uses overridden method
50     }
51 }
```

Listing 5: Default Methods in Interfaces

## 4.2 Static Methods in Interfaces

- Called directly using the **interface name**
- Not inherited by implementing classes
- Useful for utility methods related to the interface

```
1 interface CSPITStudentServices {
2     // Static method - Java 8 feature
3     static boolean validateStudentId(String studentId) {
4         return studentId != null && studentId.matches("CSPIT\\d{7}");
5     }
6
7     // Another static method
8     static String getDepartmentCode() {
9         return "CSPIT";
10    }
11 }
12
13 public class StudentValidation {
14     public static void main(String[] args) {
15         // Calling static methods using interface name
16         String studentId1 = "CSPIT2025001";
17         String studentId2 = "CSPIT2025";
18
19         System.out.println("Student ID 1 valid: " +
20             CSPITStudentServices.validateStudentId(studentId1));
21         System.out.println("Student ID 2 valid: " +
22             CSPITStudentServices.validateStudentId(studentId2));
23         System.out.println("Department: " +
24             CSPITStudentServices.getDepartmentCode());
25     }
26 }
```

Listing 6: Static Methods Usage

## 4.3 Functional Interfaces

- Contain exactly **one abstract method**
- Can have multiple default or static methods
- Can be used with lambda expressions (Java 8+)
- **@FunctionalInterface** annotation is optional but recommended

```
1 @FunctionalInterface
2 interface CSPITGradeCalculator {
3     // Single abstract method
4     String calculateGrade(double score);
5 }
```

```
5
6 // Can have default methods
7 default void displayGradeMessage(String grade) {
8     System.out.println("Final Grade: " + grade);
9 }
10
11 // Can have static methods
12 static String getGradeSystem() {
13     return "CHARUSAT Grading System (10-point scale)";
14 }
15 }
16
17 public class GradeCalculatorDemo {
18     public static void main(String[] args) {
19         // Using lambda expression
20         CSPITGradeCalculator calculator = (score) -> {
21             if (score >= 90) return "A+";
22             else if (score >= 80) return "A";
23             else if (score >= 70) return "B+";
24             else if (score >= 60) return "B";
25             else if (score >= 50) return "C";
26             else return "F (Fail)";
27         };
28
29         System.out.println(CSPITGradeCalculator.getGradeSystem())
30         ;
31         double studentScore = 85.5;
32         String grade = calculator.calculateGrade(studentScore);
33         calculator.displayGradeMessage(grade);
34
35         // Another example with different student
36         double anotherScore = 72.0;
37         String anotherGrade = calculator.calculateGrade(
38             anotherScore);
39         calculator.displayGradeMessage(anotherGrade);
40     }
41 }
```

Listing 7: Functional Interface Example

## 5 Java 9+ Features: Private Methods

### 5.1 Private Methods in Interfaces

- Introduced in Java 9
- Can be used to share common code between default and static methods
- Cannot be accessed or overridden by implementing classes

```
1 interface CSPITResearch {
2     String INSTITUTE_NAME = "CHARUSAT-CSPIT";
3
4     // Public abstract methods
5     void conductResearch();
6     void publishPaper();
7
8     // Default method using private helper
9     default void researchLifecycle() {
10         startResearch();
11         conductResearch();
12         analyzeData();
13         publishPaper();
14         endResearch();
15     }
16
17     // Private method - Java 9 feature
18     private void startResearch() {
19         System.out.println("Initiating research at " +
20             INSTITUTE_NAME);
21     }
22
23     // Private method
24     private void analyzeData() {
25         System.out.println("Analyzing research data...");
26     }
27
28     // Private method
29     private void endResearch() {
30         System.out.println("Research completed at " +
31             INSTITUTE_NAME);
32     }
33
34     // Static method with private static helper
35     static void displayResearchGuidelines() {
36         System.out.println("Research Guidelines:");
37         getEthicalGuidelines();
38         System.out.println("1. Submit research proposal");
39         System.out.println("2. Get approval from committee");
40         System.out.println("3. Conduct research ethically");
41     }
42
43     // Private static method - Java 9 feature
44     private static void getEthicalGuidelines() {
45         System.out.println("- Follow ethical research practices");
46         ;
47         System.out.println("- Ensure academic integrity");
48         System.out.println("- Acknowledge all contributors");
49     }
50 }
```

```
48
49 class CSPITResearcher implements CSPITResearch {
50     @Override
51     public void conductResearch() {
52         System.out.println("Conducting research in AI and Machine
53                             Learning");
54     }
55     @Override
56     public void publishPaper() {
57         System.out.println("Publishing paper in IEEE Conference")
58         ;
59     }
60 }
61 public class ResearchDemo {
62     public static void main(String[] args) {
63         CSPITResearch.displayResearchGuidelines();
64
65         System.out.println("\n--- Research Lifecycle
66                             Demonstration ---");
67         CSPITResearcher researcher = new CSPITResearcher();
68         researcher.researchLifecycle();
69     }
}
```

Listing 8: Private Methods in Interfaces

## 6 Extending Interfaces

### 6.1 Interface Inheritance

- One interface can inherit another using the **extends** keyword
- A class implementing the child interface must implement all methods from the entire inheritance chain
- An interface can extend multiple interfaces (unlike classes)

### 6.2 CSPIT Example: Department Hierarchy

```
1 // Base Interface - Generic Department
2 interface CSPITDepartment {
3     void displayDepartmentInfo();
4     int getTotalFaculty();
5 }
6
7 // Sub Interface - Computer Engineering Department
8 interface ComputerEngineeringDept extends CSPITDepartment {
9     void conductCodingCompetition();
10    void organizeWorkshop();
}
```

```
11 }
12
13 // Sub Interface - Information Technology Department
14 interface ITDept extends CSPITDepartment {
15     void conductHackathon();
16     void organizeTechTalk();
17 }
18
19 // Class implementing Computer Engineering Department
20 class CSPIT_CE implements ComputerEngineeringDept {
21     private int facultyCount = 25;
22
23     @Override
24     public void displayDepartmentInfo() {
25         System.out.println("Department: Computer Engineering");
26         System.out.println("Faculty Count: " + facultyCount);
27         System.out.println("Specializations: AI, ML, Cloud
28             Computing");
29     }
30
31     @Override
32     public int getTotalFaculty() {
33         return facultyCount;
34     }
35
36     @Override
37     public void conductCodingCompetition() {
38         System.out.println("Conducting annual coding competition:
39             CodeCSPIT");
40     }
41
42     @Override
43     public void organizeWorkshop() {
44         System.out.println("Organizing workshop on Full Stack
45             Development");
46     }
47 }
48
49 // Class implementing both CE and IT Department interfaces (
50 // Multiple Interface Inheritance)
51 class CSPIT_AI_ML implements ComputerEngineeringDept, ITDept {
52     private int facultyCount = 20;
53
54     @Override
55     public void displayDepartmentInfo() {
56         System.out.println("Department: AI & ML Specialization");
57         System.out.println("Faculty Count: " + facultyCount);
58         System.out.println("Specializations: AI, ML, Deep
59             Learning");
60     }
61 }
```

```
57     @Override
58     public int getTotalFaculty() {
59         return facultyCount;
60     }
61
62     @Override
63     public void conductCodingCompetition() {
64         System.out.println("Conducting AI Coding Challenge");
65     }
66
67     @Override
68     public void organizeWorkshop() {
69         System.out.println("Organizing workshop on Neural
70             Networks");
71     }
72
73     @Override
74     public void conductHackathon() {
75         System.out.println("Conducting AI Hackathon");
76     }
77
78     @Override
79     public void organizeTechTalk() {
80         System.out.println("Organizing Tech Talk on AI
81             Applications");
82     }
83 }
84
85 // Main demonstration
86 public class DepartmentDemo {
87     public static void main(String[] args) {
88         System.out.println("=== Computer Engineering Department
89             ===");
90         CSPIT_CE ceDepartment = new CSPIT_CE();
91         ceDepartment.displayDepartmentInfo();
92         System.out.println("Total Faculty: " + ceDepartment.
93             getTotalFaculty());
94         ceDepartment.conductCodingCompetition();
95         ceDepartment.organizeWorkshop();
96
97         System.out.println("\n=== AI & ML Department ===");
98         CSPIT_AI_ML aiDepartment = new CSPIT_AI_ML();
99         aiDepartment.displayDepartmentInfo();
100        aiDepartment.conductCodingCompetition();
101        aiDepartment.conductHackathon();
102    }
103 }
```

Listing 9: Interface Inheritance in CSPIT Departments

## 7 Best Practices and Use Cases

### 7.1 When to Use Interfaces

1. **Defining Contracts:** When you need to specify behavior that multiple classes must implement
2. **Achieving Abstraction:** When you want to hide implementation details
3. **Multiple Inheritance:** When a class needs to inherit behavior from multiple sources
4. **Dependency Injection:** Interfaces make it easier to swap implementations
5. **Testing:** Interfaces facilitate mocking and unit testing

### 7.2 CSPIT Example: Integrated University System

```
1 // Core Interfaces for University Management
2 interface Person {
3     String getName();
4     int getAge();
5     String getEmail();
6 }
7
8 interface Learner {
9     void learn();
10    void submitAssignment();
11    double getGrade();
12 }
13
14 interface Educator {
15     void teach();
16     void gradeAssignment();
17     void conductLecture();
18 }
19
20 // Interface inheritance - Staff extends Person
21 interface Staff extends Person {
22     String getEmployeeId();
23     double getSalary();
24 }
25
26 // Student implements Person and Learner
27 class CSPITStudent implements Person, Learner {
28     private String name;
29     private int age;
30     private String email;
31     private String studentId;
32     private double grade;
33 }
```

```
34     public CSPLITStudent(String name, int age, String email,
35         String studentId) {
36         this.name = name;
37         this.age = age;
38         this.email = email;
39         this.studentId = studentId;
40         this.grade = 85.0;
41     }
42
43     // Person interface methods
44     @Override
45     public String getName() { return name; }
46
47     @Override
48     public int getAge() { return age; }
49
50     @Override
51     public String getEmail() { return email; }
52
53     // Learner interface methods
54     @Override
55     public void learn() {
56         System.out.println(name + " is learning Java and Data
57             Structures");
58     }
59
60     @Override
61     public void submitAssignment() {
62         System.out.println(name + " submitted assignment on time"
63             );
64     }
65
66     @Override
67     public double getGrade() { return grade; }
68 }
69
70 // Professor implements Person, Educator, and Staff
71 class CSPLITProfessor implements Person, Educator, Staff {
72     private String name;
73     private int age;
74     private String email;
75     private String employeeId;
76     private double salary;
77     private String specialization;
78
79     public CSPLITProfessor(String name, int age, String email,
80         String employeeId, double salary, String
81             specialization) {
82         this.name = name;
83         this.age = age;
84         this.email = email;
```

```
81         this.employeeId = employeeId;
82         this.salary = salary;
83         this.specialization = specialization;
84     }
85
86     // Person interface methods
87     @Override
88     public String getName() { return name; }
89
90     @Override
91     public int getAge() { return age; }
92
93     @Override
94     public String getEmail() { return email; }
95
96     // Educator interface methods
97     @Override
98     public void teach() {
99         System.out.println(name + " is teaching " +
100             specialization);
101     }
102
103     @Override
104     public void gradeAssignment() {
105         System.out.println(name + " is grading assignments");
106     }
107
108     @Override
109     public void conductLecture() {
110         System.out.println(name + " is conducting lecture on " +
111             specialization);
112     }
113
114     // Staff interface methods
115     @Override
116     public String getEmployeeId() { return employeeId; }
117
118     @Override
119     public double getSalary() { return salary; }
120 }
121
122 public class UniversityManagementSystem {
123     public static void main(String[] args) {
124         // Creating student
125         CSPITStudent student = new CSPITStudent("Amit Kumar", 20,
126             "amit.kumar@charusat.edu.in", "CSPIT2025002");
127
128         // Creating professor
129         CSPITProfessor professor = new CSPITProfessor("Dr. Meera
130             Patel", 45,
```

```
128         "meera.patel@charusat.edu.in", "FAC001", 85000.0, "  
129         Artificial Intelligence");  
  
130     // Demonstrate student activities  
131     System.out.println("=== Student Activities ===");  
132     student.learn();  
133     student.submitAssignment();  
134     System.out.println("Grade: " + student.getGrade());  
135  
136     // Demonstrate professor activities  
137     System.out.println("\n=== Professor Activities ===");  
138     professor.teach();  
139     professor.conductLecture();  
140     professor.gradeAssignment();  
141     System.out.println("Employee ID: " + professor.  
142         getEmployeeId());  
143     System.out.println("Salary:      " + professor.getSalary());  
144     ;  
}
```

Listing 10: Complete University System Using Interfaces

## 8 Summary and Key Takeaways

### 8.1 Important Points to Remember

- **Interface Variables:** All variables are **public static final**
- **Interface Methods:** All methods are **public abstract** (until Java 8)
- **Default Methods:** Added in Java 8, provide default implementation
- **Static Methods:** Added in Java 8, called using interface name
- **Private Methods:** Added in Java 9, for internal interface use
- **Functional Interfaces:** Single abstract method, works with lambdas
- **Multiple Inheritance:** Only possible through interfaces
- **Keyword:** **implements** for classes, **extends** for interfaces

### 8.2 CSPIT Implementation Checklist

1. Identify core behaviors that multiple classes should share
2. Define interfaces for these behaviors
3. Implement interfaces in specific classes
4. Use interfaces in your code for flexibility

5. Leverage default, static, and private methods for code reuse
6. Apply functional interfaces for modern Java features

### 8.3 Common Mistakes to Avoid

- **Forgetting** to implement all abstract methods
  - Using **new** keyword to instantiate interfaces (not allowed)
  - Overriding default methods incorrectly
  - Confusing **extends** (for interfaces) and **implements** (for classes)
  - Not using **@Override** annotation for clarity
- 

### References and Further Reading

1. Oracle Java Documentation: <https://docs.oracle.com/javase/tutorial/java/IandI/createinterface.html>
  2. Java: The Complete Reference by Herbert Schildt
-